COMP4620/8620 – Advanced Topics in AI: Intelligent Robotics

Semester-2 2026 – Assignment 2
Convenor & Lecturer: Hanna Kurniawati
Due date: Monday, 26 October 2026 09.00 am Canberra time
Grace period: 12 hours after the due date (ie, 26 October 2026 09.00 am – 09.00pm Canberra time)
Late submission: NOT permitted
Submission after the end of the grace period will automatically receive a mark of 0.

You must read the following notes first before starting to work on the assignment.

1. This is an individual assignment with a maximum total mark of 100 points and worth 35% of your total mark in this class.
2. This assignment consists of three parts: Part A, Part B, and in-person Q&A. The maximum mark for Part A is 30 points. The maximum mark for Part B is 35 points. The in-person Q&A has two components: (1) Demo of your program (B3). (2) Oral Q & A related to the assignment and all materials covered in Week 4 to Week 8 (inclusive). The maximum mark for component (2) of the in-person Q&A is 35 points.
3. The use of Generative AI is allowed, but you must disclose it and provide all the prompts used. Failure to disclose constitute cheating.
4. Submission Instruction for Part A and Part B:
   (a) You must submit the following 2 files via Canvas before the due date.
      i. One .pdf file containing answers to all the non-programming part of the assignment
      ii. One .zip/.7z/.tar.gz file containing your source codes and test cases.
   (b) For the Programming component, you must:
      i. Write your program in Python. We provide a set of functionalities you can use to help in this assignment. These functionalities are written in Python.
      ii. Submit all of your source codes and test cases used in your report. You must place all files (and folders) under a single top-level folder, compress the top-level folder into a single file with one of the following extensions: .zip or .7z or .tar.gz, and submit the compressed file. If your program consists of multiple files in multiple folders, your compressed file should preserve the folder structure. Note that during the in-person Q&A, you must download this file from Canvas, unzip, and run them in the lab's computer to solve the test scenarios we provide. Hence, all the files you need to run your program, aside from the libraries already installed in the lab's computer, must be included in this file.
   (c) Late submission is NOT permitted. We do provide a 12 hours grace period. Within the grace period, you can still submit and resubmit your assignment. However, after the grace period ends, you will NOT be able to submit your assignment. Without a submission, you will receive a 0 mark for this assignment.
5. Information for the in-person Q&A:
   (a) The in-person Q&A will take place on Friday, 30 October 2026 in Skaidrite Darius Building Room N114. The Schedule is in https://bit.ly/comp4620A2QASched . If your name is not listed in the schedule, you must contact the teaching staff via Ed Forum of the class.
   (b) You are recommended to arrive at least 10 minutes before the start of your Q&A slot.
   (c) You must bring your ANU ID and show it to the staff marking your Q&A.
   (d) You will not be able to participate in the in-person Q&A if you do not submit Part A and Part B before the end of the grace period (see point 4 above).

PART A [30 pts]

1. [10 pts]

   Consider a simple robot navigation on a 1-D grid cell, discretised into k cells (illustrated in Figure 1). Suppose the robot starts from cell-1 and needs to reach cell-k. At each step, the robot can move to the left or to the right. It ends up in the immediate neighbor in the intended direction with probability 0.7 and in the immediate neighbor in the opposite direction with probability 0.3. If a move would take the robot beyond the end of the cell, the robot stays in its current cell instead. For example, moving left from cell-1 keeps tit in cell-1 with probability 0.7 and takes it to cell-2 with probability 0.3.

   Figure 1: Illustration of the grid cell environment.

   The navigation problem is framed as an MDP where the state space S is the cells, ie, S = {cell-1, cell-2, $\cdots$, cell-k}, the action space is A = {moveLeft, moveRight}, and a transition function tht captures the motion uncertainty described above. Each move incurs a reward of -1 and reaching cell-k gives a reward of $3k$.

   Mr A said that when k is large (e.g., $> 1,000$), the above MDP problem should be abstracted into an MDP with just 2 states, where the state space becomes $S' = \{L, R\}$, with L representing cell-1 to cell-$\lceil \frac{k}{2} \rceil$ and R represents cell-$(\lceil \frac{k}{2} \rceil + 1)$ to cell-k. Is Mr A correct? In particular, consider the transition function property of MDP and whether it can be satisfied while still capturing the original motion uncertainty. If Mr A is correct, please provide the abstracted MDP problem. Otherwise, please explain why such an abstraction is wrong.

2. [10 pts] Recall the α-vector policy representation in Offline POMDP Solving. Suppose P is two-state POMDP with the state space $S = \{s_1, s_2\}$ and suppose the following is the set of α-vectors $\Gamma = \{\alpha_1 = [10, -5],\ \alpha_2 = [-5, 10],\ \alpha_3 = [4, 4],\ \alpha_4 = [1, 2],\ \alpha_5 = [5.5, 1]\}$. Then:

   (a) [4 pts] Provide the belief intervals on which each α-vector in Γ is maximal.

   (b) [6 pts] Which α-vector(s) in Γ can be removed by pointwise dominance? An α-vector $\alpha \in \Gamma$ can be removed by pointwise dominance whenever $\forall_{b \in B} \exists_{\alpha' \in \Gamma}\ \alpha \cdot b < \alpha' \cdot b$).

3. [10 pts]

   Consider the problem of pedestrian avoidance when a self-driving car moving on a single-lane road (illustrated in Figure 2). We can assume the self-driving car has good enough sensors to identify its own position and the pedestrians' positions. This pedestrian avoidance problem can be framed either as an MDP or as a POMDP under different scenarios. Provide a scenario in which MDP would be a suitable framing, and a scenario in which POMDP is a suitable framing. In each scenario, explain and elaborate why MDP/POMDP would be the more suitable framing.

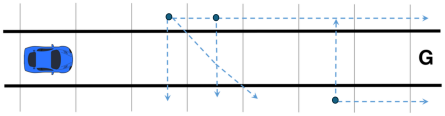
   

   Figure 2: Illustration of the pedestrian avoidance problem. The notation "G" indicates the car's destination. The small blue discs illustrate positions of pedestrians, while the arrows illustrate potential destinations of the pedestrians.

PART B [35 pts]

In this part, you are asked to perform a basic implementation, experiment, analysis and reporting that one generally does as part of a research work and paper writing in robot planning under uncertainty. However, of course the problem in this assignment has been significantly simplified to fit the assignment timeline.

Throughout this part of the assignment, consider the problem of centralised collision-free navigation of multi-drones in the presence of non-deterministic action effects, such as in the presence of wind disturbances. To simplify the problem, assume that the 3D environment is discretized into uniform grid cells of size $n_x \times n_y \times n_z$. Some of these cells are marked as obstacles. At each time step, each drone occupies a cell, and will make a decision of where to move next. A drone that occupies an obstacle cell is in collision, and two or more drones are in collision when they occupy the same cell. Collision will cause the mission to terminate. Each drone are initially located in one of the cells and has a goal cell to go to. Assuming that information about uncertainty of the effects of actions is provided as a probability function, how should the drones move, so that they can reach their goal cells as fast as possible without colliding with obstacles and each other? This part of the assignment explores MDP modelling of the above navigation problem and on-line solving of the MDP. To this end, you need to answer the following questions.

1. [3 pts] Define an MDP framing of the problem, assuming in a single step, a drone can move from a cell to one of the 26 neighbours of the cell and the uncertainty of the action effects are given as a probability function T_{known}.
2. [4 pts] Design a specific approximate on-line MDP solver and provide the pseudo-code. This means, if for instance you use MCTS, you need to specify the exact method you use for each of its four components.
3. [8 pts] Implement your pseudo-code, utilising primitive functionalities we provide (downloadable from `https://github.com/RDLLab/MultiDroneUnc`). The mark for this question will come from the program's performance during the demo. Specifically, there will be 2 testing scenarios, ranging from a problem with only 1 drone that does not change altitude operating in a relatively open environment, to a problem with up to 4 drones that may change altitude operating in a cluttered environment with up to 5 obstacles. Your program has up to 10 seconds planning time per step with no parallelisation to solve a given problem. Each solved problem is worth 4 points.
   The test case we expect you to do well on will have less than 50 discrete actions, but we do have a challenge problem, which we can give if you are interested :)
4. [20 pts] Empirically investigate how the performance of the MDP solver you proposed changes as the problem becomes more complex. Here, complexity is characterised by three factors: the size of the state space, the size of the action space, and the degree of clutter in the environment. Design a systematic experiment to determine which of these factors degrades the performance of your solver the most. To do so, design several sets of testing scenarios and explain what you expect to learn from each set. Run each scenario multiple times and report summary statistics, including at least the mean and 95% confidence interval. Finally, discuss the results, including how each complexity factor affects the performance of your solver, and how each component of your solver affects its performance as the problem becomes more complex.

Note that unlike Part A, the questions in this part are inter-related. For instance, your answer to question 3 must be an implementation of your answers to question 1 and 2, and your answers to questions 4 must be based on the results of running the code you implemented to answer question 3.

That's all folks